

Автоматизация итеративного ревью кода с помощью LLM-комментариев для снижения числа итераций исправления

А. К. Белоногов А. А. Лаушкина В. Д. Волоха И. З. Смирнов

2026

Abstract

При разрешении pull request комментарий ревью полезен не сам по себе, а только если он помогает быстрее перейти к исправлению. Такой комментарий должен быть связан с внесенным изменением, указывать на конкретную проблему и уменьшать число последующих итераций правки. В работе рассматривается задача автоматического формирования таких комментариев для снижения числа итераций при разрешении pull request.

Для решения этой задачи разработан инструмент MergeMind. Он формирует один итоговый комментарий к изменению кода: генератор предлагает возможный комментарий, оценщик проверяет его связь с правкой и потенциальную полезность, а копирайтер переводит выбранный вариант в краткую инструкцию для исправления. Такой подход отличается от генерации набора независимых комментариев: MergeMind стремится выбрать один комментарий, который можно сразу использовать в следующей итерации работы над pull request.

Полезность комментариев проверяется на стенде SWE-CI, который используется как способ валидации, а не как основная тема работы. В экспериментах измеряются число итераций, финальный и лучший разрыв по тестам, новые непройденные тесты и стоимость дополнительных обращений к модели. Пробные запуски показали смешанный результат: на задаче `cle-b/httpdbg` финальный разрыв по тестам снизился с 11 до 5, однако на `inline-snapshot` финальный результат не улучшился, а проверка на четырех задачах не подтвердила устойчивого сокращения числа итераций. Следовательно, MergeMind показывает локальный положительный эффект, но гипотеза о стабильном снижении числа итераций при разрешении pull request требует дальнейшей проверки.

1 Введение

Современные модели позволяют автоматически формировать комментарии по изменению кода, однако практические исследования показывают, что такие комментарии часто бывают шумными: они могут быть слишком общими, плохо связанными с дефектом или непригодными для дальнейшего исправления [8, 7]. В данной работе практическая ценность комментария определяется не тем, похож ли он на человеческий текст, а тем, помогает ли он быстрее разрешить pull request: указать на конкретную проблему в изменении, направить следующую правку, уменьшить число непройденных тестов или хотя бы не ухудшить уже найденное частичное решение. Поэтому в работах о полезных комментариях ревью отдельно подчеркивается необходимость отбирать полезные и обоснованные комментарии, а не просто увеличивать их количество [7].

В этой статье автоматический комментарий рассматривается не как финальный ответ для человека, а как средство управления итеративным исправлением кода. Такой сценарий отличается от обычной генерации ревью: важно не только качество текста, но и то, меняется ли поведение агента после получения комментария. Если комментарий действительно полезен, агент должен быстрее прийти к рабочему варианту, удержать меньше падающих тестов или хотя бы не ухудшить уже найденное частичное решение.

Проверять такую гипотезу удобно в среде, где каждая попытка исправления измеряется тестами. В работе используется SWE-CI [12]: агент получает требование к изменению, меняет код, а результат оценивается запуском тестов в цикле непрерывной интеграции. В отличие от статических наборов задач на исправление кода, такой стенд позволяет наблюдать траекторию решения: сколько итераций выполнил агент, как менялось число падающих тестов и не появлялись ли новые регрессии.

Исследовательский вопрос работы формулируется следующим образом: может ли один автоматически сформированный комментарий ревью, переданный агенту после первой правки, улучшить дальнейший ход исправления? Из него выделяются три проверяемые гипотезы:

- **H1:** комментарий уменьшает число итераций, необходимых агенту для достижения лучшего наблюдаемого результата;
- **H2:** комментарий уменьшает финальное или лучшее число непройденных тестов по сравнению с запуском без MergeMind;
- **H3:** комментарий не должен давать улучшение только ценой неконтролируемых дополнительных правок и существенного роста стоимости запуска.

Вклад работы состоит в разработке и первичной проверке инструмента MergeMind, направленного на снижение затрат при исправлении ошибок, выявленных в ходе разрешения pull request. Основная идея состоит в том, что более полезный комментарий к изменению кода может сократить число последующих итераций правки. Инструмент реализован как алгоритм из трех этапов: генератор формирует комментарий к изменению, оценщик проверяет его связь с правкой и потенциальную полезность, а копирайтер переводит выбранный комментарий в краткую инструкцию для исправления. В экспериментах проверяется, меняет ли такой комментарий дальнейший ход исправления по сравнению с запуском без MergeMind.

2 Связанные работы

Автоматические комментарии ревью. Для данной работы важны не все подходы к автоматическому ревью, а те, которые различают полезные и бесполезные комментарии. Комментарий ревью может быть сформулирован корректно и выглядеть похожим на человеческое комментарий, но при этом не помогать дальнейшему исправлению кода. Работы о желательных комментариях показывают, что не каждое историческое комментарий одинаково пригодно для обучения и оценки модели [7]. Практические исследования автоматического ревью также указывают на проблему шума: автоматически сгенерированный комментарий может быть слишком общим, плохо связанным с дефектом или непригодным для конкретной правки [8].

Из этого следует ограничение для MergeMind: система не должна генерировать много комментариев ради полноты. В данной постановке нужен один комментарий, который связан с текущим изменением и может быть использован агентом в следующей итерации исправления.

LLM-агенты для исправления программ. Другое близкое направление связано с агентами, которые исправляют ошибки в реальных репозиториях. SWE-bench стал одним из основных наборов задач для оценки таких агентов на GitHub-задачах [1]. SWE-agent показывает, что результат зависит не только от языковой модели, но и от организации взаимодействия агента с окружением разработки [2]. Agentless демонстрирует, что часть задач можно решать без сложного агентного цикла, если аккуратно построить локализацию и исправление [3]. Reflexion исследует самокоррекцию через текстовую обратную связь после неудачных попыток [4].

MergeMind связан с этим направлением, но проверяет другой объект. В центре работы находится не способность агента самостоятельно решить задачу, а влияние внешнего комментария ревью на следующую итерацию исправления. Поэтому комментарий рассматривается как управляющий сигнал внутри агентного цикла.

Оценка кода и комментариев. Для задач генерации и исправления кода часто используются метрики сходства с эталонным решением, например CodeBLEU [5]. Такие метрики полезны, когда нужно сравнить сгенерированный код с известным правильным решением, но они плохо отвечают на вопрос о пользе комментария ревью. Полезный комментарий может не совпадать с историческим комментарием по тексту, но все равно приводить к правильному исправлению.

Другой подход состоит в использовании LLM-as-a-judge, когда языковая модель оценивает комментарий по заданной рубрике: обоснованность, полезность, конкретность и применимость [6]. Такой подход ближе к задаче оценки ревью, но он все равно оценивает преимущественно текст. В данной работе основная проверка переносится в цикл исполнения: комментарий считается полезным только тогда, когда после него улучшается траектория исправления, уменьшается число падающих тестов или не появляются новые регрессии.

SWE-CI как экспериментальный стенд. SWE-CI проверяет агента в итеративном цикле непрерывной интеграции: агент получает требование, меняет код, запускает тесты и продолжает исправление по результатам проверки [12]. Поэтому этот стенд подходит для данной работы лучше, чем оценка только по тексту комментария или сходству с эталонным кодом. Он позволяет измерять не только финальное состояние решения, но и траекторию: число итераций, лучший достигнутый разрыв по тестам, финальный разрыв, новые падения и стоимость дополнительных обращений к модели.

Таким образом, MergeMind занимает промежуточное положение между автоматическим ревью кода и агентным исправлением программ. От первого направления берется идея комментария по изменению, от второго — итеративная среда исправления и проверка через тесты. Основной вопрос работы состоит в том, может ли один автоматически выбранный комментарий ревью улучшить следующую итерацию агента.

3 Постановка задачи

Рассматривается итеративное исправление кода агентом-разработчиком. На вход агенту подается требование к изменению и текущее состояние репозитория. Агент делает правку, после чего результат проверяется тестами. Если задача не решена, агент продолжает исправление в следующей итерации.

В этой работе итерацией называется один цикл между двумя запусками тестов: агент получает требование и контекст, изменяет код, при необходимости получает внешний комментарий ревью, выполняет дополнительную правку и после этого запускает тесты. Отдельный вызов модели или отдельное изменение файла не считается самостоятельной итерацией, если после него не был выполнен запуск тестов.

Пусть g_i обозначает число падающих тестов после i -й итерации. Тогда задача состоит в том, чтобы проверить, может ли комментарий ревью, переданный после первой правки агента, улучшить дальнейший ход исправления: уменьшить число итераций до лучшего результата, снизить финальный или лучший разрыв по тестам или предотвратить появление новых падений.

Комментарий ревью в данной постановке не является самостоятельным ответом для человека. Он рассматривается как управляющий сигнал для агента. Такой комментарий должен быть основан только на видимом требовании, текущей правке, доступном контексте репозитория и уже наблюдаемых результатах тестов. Скрытое правильное исправление и эталонная разница с ним не используются.

Таким образом, в постановке задачи сравниваются два режима: обычный запуск агента без внешнего комментария и запуск, где после первой правки агент получает один автоматически выбранный комментарий ревью. Способ формирования этого комментария описан в следующем разделе.

4 Метод

Метод строится из ограничения, заданного в постановке задачи: агенту нельзя передавать скрытое правильное решение, но можно дать один комментарий ревью, основанный на уже сделанной первой правке. Поэтому MergeMind рассматривается не как самостоятельный решатель задачи, а как дополнительный модуль обратной связи внутри итерации SWE-CI.

4.1 Входные данные MergeMind

В каждой итерации сначала работает агент-разработчик: он получает требование задачи и делает первую правку кода. Только после этой правки запускается MergeMind. На вход MergeMind передаются:

- текст требования к текущей задаче;
- список файлов, измененных агентом в первой правке;
- видимая разница между состоянием этих файлов до и после первой правки;
- доступный контекст текущего репозитория;
- уже наблюдаемые результаты предыдущих запусков тестов, если они есть.

MergeMind не получает скрытое правильное исправление, не получает `target_sha`, не получает эталонную разницу с правильным решением и не получает файлы из будущего состояния репозитория. Это ограничение реализуется на уровне подготовки запроса: модуль, который формирует вход MergeMind, собирает только данные из текущего рабочего каталога агента и из уже наблюдаемых логов запуска. Поля, связанные с эталонным исправлением, в запрос не включаются.

4.2 Цепочка генератор–оценщик–копирайтер

MergeMind состоит из трех этапов: генератора, оценщика и копирайтера.

Генератор получает требование, первую правку агента и доступный контекст. Его задача – предложить возможные комментарии к изменению. На этом этапе комментарий

еще не считается пригодным для передачи агенту: генератор может сформулировать слишком общий, слабо связанный или потенциально вредный совет.

Оценщик проверяет черновой комментарий. Комментарий принимается только если он удовлетворяет трем условиям:

- он связан с конкретным измененным фрагментом кода;
- он указывает на возможную ошибку поведения программы, а не на общий стиль или произвольное улучшение;
- по нему можно сделать ограниченное исправление в уже измененных файлах.

Если оценщик не находит достаточной связи между комментарием, требованием, первой правкой и наблюдаемыми тестами, комментарий отклоняется. В этом случае агент продолжает обычный запуск без подсказки MergeMind.

Копирайтер получает только принятый комментарий и переводит его в короткую инструкцию для агента-разработчика. Инструкция должна быть пригодна для следующей правки: она кратко указывает, какую проблему нужно проверить и в каком направлении исправлять код. Копирайтер не генерирует готовый патч и не добавляет информацию, которой не было во входных данных MergeMind.

4.3 Передача комментария агенту

На выходе MergeMind формирует не список комментариев, а не более одного итогового комментария за итерацию. Это сделано для того, чтобы проверять влияние одного выбранного сигнала, а не эффект большого дополнительного контекста.

Если комментарий принят, он сохраняется как отдельный файл с инструкцией для агента. Вместе с комментарием агент получает список файлов, которые разрешено менять в дополнительной правке. Обычно этот список совпадает с файлами, измененными в первой правке агента. Если агент пытается изменить файлы вне разрешенного списка, такая дополнительная правка не принимается.

После этого агент выполняет дополнительную правку с учетом комментария MergeMind. Только после этой дополнительной правки запускаются тесты SWE-CI. Если MergeMind не выдал комментарий, дополнительная правка не выполняется, и запуск продолжается как обычная итерация SWE-CI.

4.4 Технические ограничения

Чтобы комментарий не использовал скрытое решение, в реализации соблюдаются следующие ограничения:

- в запрос MergeMind не передается `target_sha`;
- в запрос не передается эталонная разница с правильным исправлением;
- MergeMind анализирует только первую правку агента, а не будущие состояния репозитория;
- результаты тестов используются только в том случае, если они уже были реально получены в предыдущих итерациях;
- дополнительная правка агента ограничивается списком разрешенных файлов.

Таким образом, MergeMind не подсказывает агенту готовое решение задачи. Он может только сформулировать комментарий по уже видимому изменению и передать его как ограниченную обратную связь перед запуском тестов.

4.5 Что фиксируется в эксперименте

Для последующего анализа в каждом запуске сохраняются:

- был ли сформирован комментарий MergeMind;
- был ли комментарий принят оценщиком;
- была ли выполнена дополнительная правка агентом;
- какие файлы были изменены в первой и дополнительной правке;
- сколько итераций выполнил агент;
- как менялось число падающих тестов после запусков SWE-CI;
- сколько дополнительных вызовов модели и токенов потребовал MergeMind.

Эти данные позволяют разделить исходы запуска на три группы:

- **полезный комментарий** — после комментария уменьшается число падающих тестов, раньше достигается лучший разрыв или исчезают финальные падения базового режима;
- **нейтральный комментарий** — комментарий был передан агенту, но число итераций, лучший разрыв и финальный разрыв не улучшились;
- **вредный или слишком дорогой комментарий** — после дополнительной правки появились новые падения, ухудшился разрыв по тестам или улучшение отсутствует при дополнительных вызовах модели и росте числа токенов.

5 Экспериментальная схема

Все результаты в статье относятся только к SWE-CI. Базовый режим запускает агента без MergeMind. Режим с MergeMind добавляет шаг автоматического ревью и дополнительной правки перед запуском тестов. Для честного сравнения в каждой паре запусков используются одинаковые задачи, одинаковый лимит итераций и одна и та же локальная модель `qwen3.6-27b@iq2_xxs`, запущенная через LM Studio. Эта модель используется в обоих режимах; различие между режимами состоит только в том, получает ли агент дополнительный комментарий MergeMind перед запуском тестов.

Используются следующие метрики:

- **EvoScore** — официальный показатель SWE-CI из вывода стенда;
- **число итераций** — сколько итераций выполнил агент;
- **траектория разрыва** — как менялось число падающих тестов по ходу запуска;
- **финальный разрыв (final gap)** — число падающих тестов в последней итерации;
- **лучший разрыв (best gap)** — минимальное неотрицательное число падающих тестов за весь запуск;
- **валидный финал** — признак того, что финальный запуск тестов был корректным;

- **исправленные и новые падения** — сколько финальных падений базового режима исчезло и сколько новых появилось в режиме с MergeMind.
- **стоимость комментария** — сколько дополнительных вызовов модели и токенов потребовал MergeMind.

Значение `final_gap=-1` не считается улучшением. Оно означает, что `pytest` не был корректно выполнен или результат тестов нельзя интерпретировать как обычное число падающих тестов.

6 Результаты

6.1 Пробные запуски SWE-CI

В таблице 1 приведены пробные сравнения двух режимов. Они показывают, в каких случаях комментарий MergeMind менял ход запуска, а в каких не давал улучшения по основным метрикам.

Строка “С MergeMind” означает, что после первой правки агент получил один комментарий, выбранный оценщиком MergeMind, и сделал дополнительную правку перед запуском тестов.

Table 1: SWE-CI: пробные сравнения без MergeMind и с MergeMind

Запуск	EvoScore	итерации	фин. разрыв	лучш. разрыв
Без MergeMind, <code>inline-snapshot</code>	0.5045	3	12	12
С MergeMind, <code>inline-snapshot</code>	0.8333	3	12	12
Без MergeMind, <code>cle-b/httpdbg</code>	-0.5439	3	11	5
С MergeMind, <code>cle-b/httpdbg</code>	0.0000	3	5	5

На задаче `inline-snapshot` MergeMind повысил официальный EvoScore, но не уменьшил число итераций, финальный разрыв и лучший разрыв.

На задаче `cle-b/httpdbg` результат сильнее: базовый режим завершился с финальным разрывом 11, а режим с MergeMind — с финальным разрывом 5. В этом запуске MergeMind устранил 6 финальных падений базового режима и не добавил новых финальных падений. Однако и здесь число итераций не снизилось: оба режима использовали заданный лимит из трех итераций.

6.2 Траектория разрыва по тестам

Для вопроса о скорости сходимости важно смотреть не только финальную строку, но и траекторию разрыва по тестам. В таблице 2 показано, как менялось число падающих тестов по ходу запуска.

Table 2: SWE-CI: траектории разрыва в пробных запусках

Запуск	Траектория разрыва
Без MergeMind, <code>inline-snapshot</code>	16 → 106 → 13 → 12
С MergeMind, <code>inline-snapshot</code>	16 → 13 → 13 → 12
Без MergeMind, <code>cle-b/httpdbg</code>	5 → -1 → 5 → 11
С MergeMind, <code>cle-b/httpdbg</code>	5 → 5 → 5 → 5

На `inline-snapshot` запуск с MergeMind не допустил резкого ухудшения с 16 до 106 падающих тестов, но к финалу пришел к тому же разрыву 12, что и базовый запуск. На `cle-b/httpdbg` режим с MergeMind удержал разрыв 5 на всех итерациях, тогда как базовый запуск завершился разрывом 11.

6.3 Проверка на нескольких задачах

Чтобы проверить, не является ли результат на `cle-b/httpdbg` единичным, был проведен четырехзадачный запуск с более строгой проверкой комментариев. Этот запуск не показал улучшения: среднее число итераций осталось 3.000 против 3.000, средний лучший разрыв — 6.250 против 6.250, а все финальные запуски в обеих группах были невалидными. Среднее изменение официального EvoScore составило -0.070.

Стоимость этого запуска также выросла: MergeMind потратил 135610 токенов на ревью и 32 дополнительных вызова модели. Это число относится ко всему четырехзадачному запуску, а не к одной задаче.

7 Обсуждение

Результаты следует интерпретировать относительно гипотез, сформулированных во введении.

H1: сокращение числа итераций. Эта гипотеза пока не подтверждена. В завершенных сравнимых запусках агент использовал заданный лимит итераций как без MergeMind, так и с MergeMind. На `inline-snapshot` режим с MergeMind раньше избежал резкого ухудшения разрыва по тестам, а на `cle-b/httpdbg` удержал лучший разрыв до финала. Однако это не является сокращением числа итераций в строгом смысле.

H2: улучшение результата по тестам. Эта гипотеза получила частичное подтверждение. На `cle-b/httpdbg` финальный разрыв снизился с 11 до 5, а 6 финальных падений базового режима исчезли без появления новых финальных падений. На `inline-snapshot` финальный и лучший разрывы не улучшились, хотя траектория стала стабильнее. На четырехзадачной проверке средний лучший разрыв также не улучшился.

H3: стоимость и безопасность комментария. Эта гипотеза остается ограничением метода. Комментарий полезен только тогда, когда он приводит к измеримому улучшению или предотвращает ухудшение. Если комментарий слабый, он может добавить дополнительные вызовы модели и токены без выигрыша по тестам. Поэтому стоимость комментария должна учитываться вместе с качеством решения, а не отдельно от него.

Главное ограничение текущей работы — малый размер выборки SWE-CI. На текущих данных можно утверждать только, что MergeMind технически встраивается в цикл SWE-CI и иногда улучшает отдельные траектории. Устойчивое улучшение и сокращение числа итераций пока не доказаны.

8 Заключение

В работе рассмотрен прототип MergeMind для автоматического комментария ревью внутри итеративного исправления кода. Комментарий используется не как самостоятельный ответ для человека, а как управляющий сигнал для следующей правки агента. Метод формирует не более одного комментария за итерацию с помощью генератора, оценщика и копирайтера.

Первичные эксперименты показывают, что такой комментарий может улучшать отдельные траектории SWE-CI, но пока не дает устойчивого выигрыша на наборе задач. Поэтому следующий этап работы должен быть направлен на расширение выборки, более строгий отбор комментариев и ручной анализ случаев, где комментарий помогает, не влияет на результат или ухудшает запуск.

References

- [1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770, 2023. <https://arxiv.org/abs/2310.06770>
- [2] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, O. Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793, 2024. <https://arxiv.org/abs/2405.15793>
- [3] X. Xia, Y. Deng, S. Dunn, L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024. <https://arxiv.org/abs/2407.01489>
- [4] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366, 2023. <https://arxiv.org/abs/2303.11366>
- [5] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, S. Ma. CodeBLEU: A Method for Automatic Evaluation of Code Synthesis. arXiv:2009.10297, 2020. <https://arxiv.org/abs/2009.10297>
- [6] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, C. Zhu. G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. EMNLP, 2023. <https://aclanthology.org/2023.emnlp-main.153/>
- [7] Yu Y. et al. Distilling Desired Comments for Enhanced Code Review with Large Language Models. arXiv:2412.20340, 2024. URL: <https://arxiv.org/abs/2412.20340>.
- [8] Automated Code Review In Practice. arXiv:2412.18531, 2024. URL: <https://arxiv.org/abs/2412.18531>.
- [9] Cheng X., Wang H., Hua J., Xu G., Sui Y. Towards Practical Defect-Focused Automated Code Review. arXiv:2505.17928, 2025. URL: <https://arxiv.org/abs/2505.17928>.
- [10] CodeReviewQA: The Code Review Comprehension Assessment for Large Language Models. arXiv:2503.16167, 2025. URL: <https://arxiv.org/abs/2503.16167>.
- [11] CoDocBench: A Dataset for Code-Documentation Alignment in Software Maintenance. arXiv:2502.00519, 2025. URL: <https://arxiv.org/abs/2502.00519>.
- [12] SWE-CI: Evaluating Agent Capabilities in Maintaining Codebases via Continuous Integration. arXiv:2603.03823, 2026. URL: <https://arxiv.org/abs/2603.03823>.
- [13] METAMON: Finding Inconsistencies between Program Documentation and Behavior using Metamorphic LLM Queries. arXiv:2502.02794, 2025. URL: <https://arxiv.org/abs/2502.02794>.
- [14] Tree-sitter: An incremental parsing system for programming tools. URL: <https://github.com/tree-sitter/tree-sitter>.